

Coordination

ECE 454: Distributed Computing

Instructor: Dr. Tahsin Reza

tahsin.reza@uwaterloo.ca

Coordination

- Majority of the following slides are adapted from M. Van Steen and A. S. Tanenbaum, Distributed Systems (textbook).
- Most of the topics are discussed in Chapter 5 of the textbook.

Overview

A distributed system must be “available” for it to function properly.

- A valid response to a request from a client in a timely manner
- Correct result in a computation task
- Data integrity and fault tolerance
- Fairness in resource sharing, deadlock avoidance

Ensuring availability requires effective and efficient “coordination” between components of a distributed system.

Coordination

The goal is to manage the interactions and dependencies between activities in a distributed system.

- Leader election
 - Selection of a server or process as the coordinator
- Mutual exclusion (locks)
 - Exclusive access to shared resources and critical regions of programs
- Membership in a process group
- Barrier synchronization, quiescence/termination detection
- Clock synchronization
 - Lamport's logical clocks
- Ordering of events (transactions), event matching
 - E.g., to ensure sequential consistency
- Overlay topology construction
- Naming
- Notification

Leader election

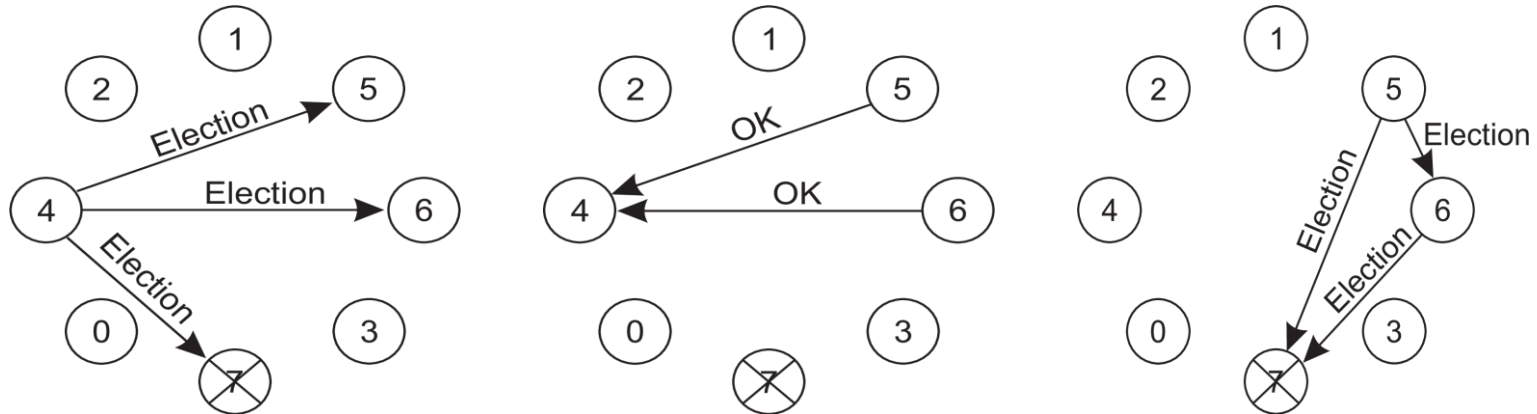
- A distributed application may require that at least one process acts as a coordinator, initiator etc.
- The goal of an election algorithm is to ensure that when an election starts, it concludes with all processes agreeing on whom the new coordinator is going to be.
- The problem to solve is how to select a coordinator, a leader process dynamically.

Assumption (in the following examples):

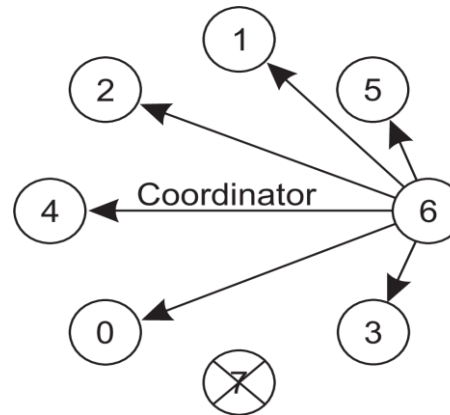
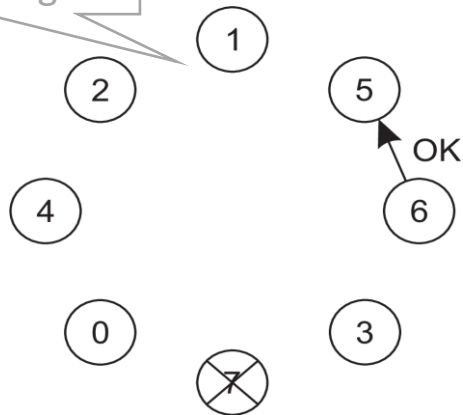
- All processes have unique ID's.
- All processes know ID's of all processes in the system (but not whether they are up or down).
- An election algorithm identifies the process with the highest ID that is up in a distributed manner.

Leader election: the bully algorithm

(Section 5.4.1, in the textbook)



5 stops sending
Election msg.



Three types of messages:
1. Election
2. OK
3. Coordinator

6 never received OK. 6
becomes Coordinator,
lets others know

Leader election: the bully algorithm

Consider N processes $\{P_0, \dots, P_{N-1}\}$ and let $id(P_k) = k$. When a process P_k notices that the coordinator is no longer responding to requests, it initiates an election:

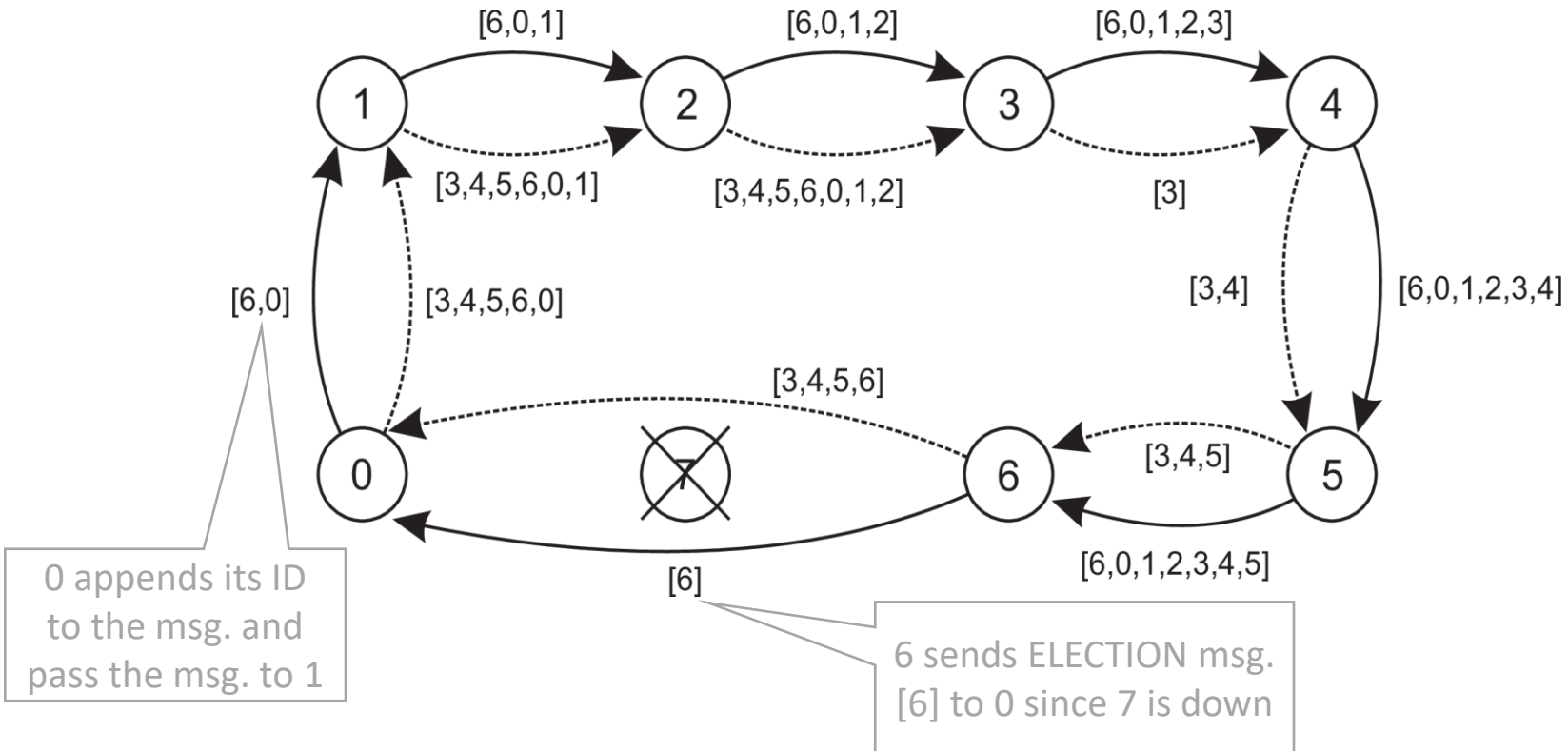
- P_k sends an ELECTION message to all processes with higher identifiers: $P_{k+1}, P_{k+2}, \dots, P_{N-1}$.
- If no one responds, P_k wins the election and becomes coordinator.
- If one of the higher-ups answers, it takes over and P_k 's job is done.

Message complexity:

$$N-1 + N-2 + \dots + 1 = (N-1)*N/2 = O(N^2)$$

Leader election: the ring algorithm

(Section 5.4.2, in the textbook)



- The solid line shows the election messages initiated by P_6
- The dashed ones, the messages by P_3

Leader election: the ring algorithm

Process priority is obtained by organizing processes into a (logical) ring. Messages propagate on the ring in one direction. The process with the highest priority (largest ID in the example) is elected as the coordinator.

- Any process can start an election by sending an ELECTION message to its successor. If a successor is down, the message is passed on to the next successor (assuming such a background protocol exists).
- If a message is passed on, the sender adds itself to the list. When it gets back to the initiator, everyone had a chance to make its presence known.
- The initiator sends a COORDINATOR message around the ring containing a list of all living processes. The one with the highest priority is elected as coordinator.

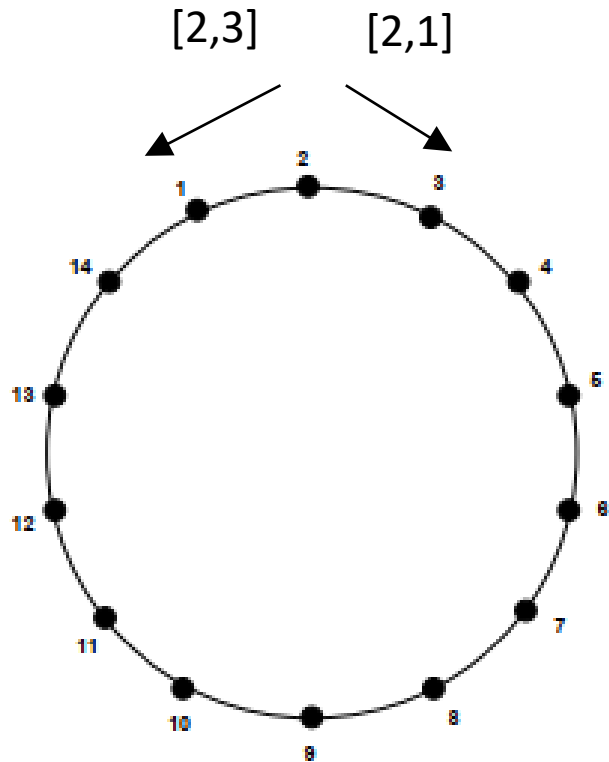
Message complexity:

$$2 \cdot (N-1) = O(N)$$

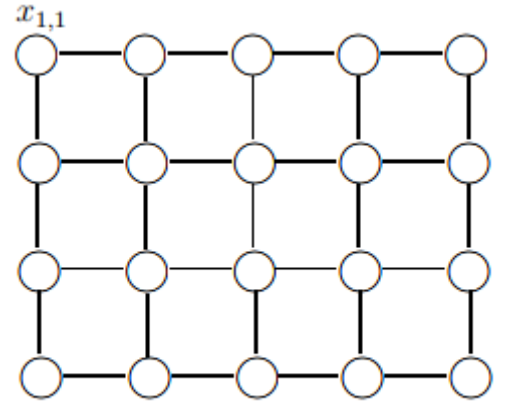
What are the tradeoffs of bully and ring algorithms?

Leader election

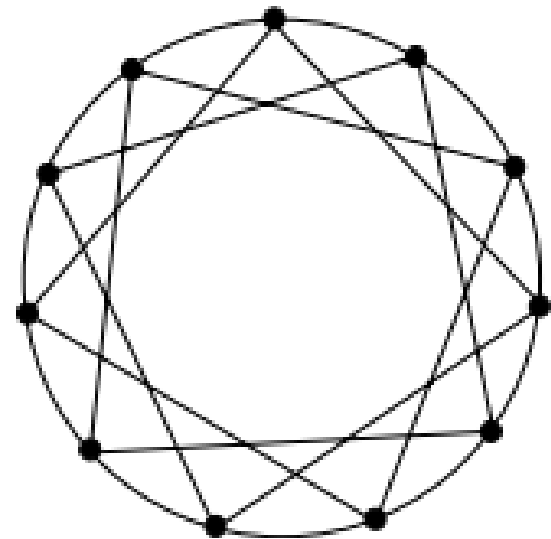
Other network topologies, messaging techniques



Bidirectional



Grid



Chordal ring

Mutual exclusion (in a distributed system)

- Several processes in a distributed system want exclusive access to some resource.
- Establish access to **critical region**, a section of code that can be executed by at most one process at a time.

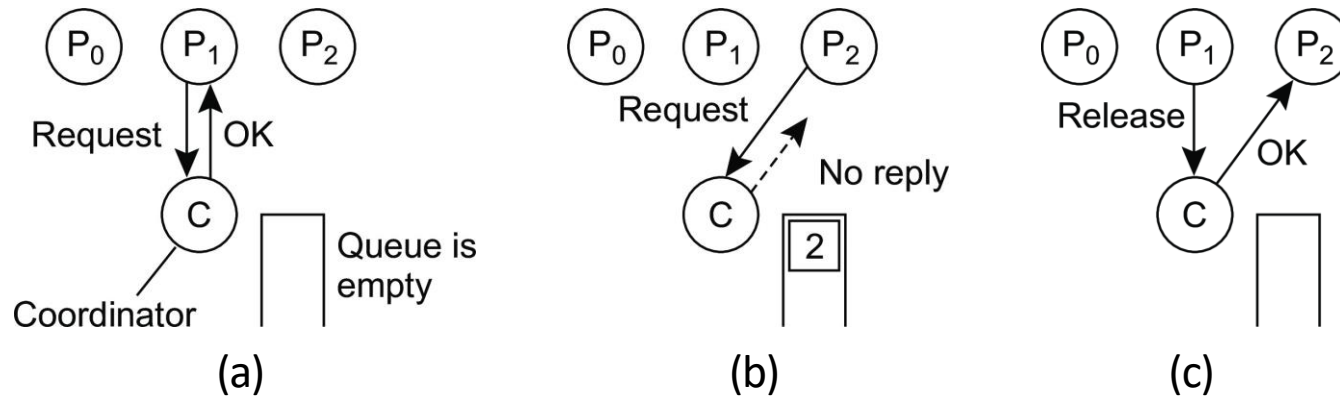
Solution approaches:

- Permission-based - A process wanting to enter a critical region, or access a resource, needs permission from other processes.
- Token-based - A token is passed between processes. The one who has the token may proceed in its critical region, or pass it on when not interested.

Mutual exclusion: centralized coordinator

(Section 5.3.2, in the textbook)

Prerequisite: A coordinator process C has been elected using a leader election algorithm. In coming requests are placed in a queue if the shared resource is in use.



- Process P_1 asks the coordinator for permission to access a shared resource. The coordinator replies OK, i.e., permission is granted.
- Process P_2 then asks permission to access the same resource. The request is placed in the queue. The coordinator does not reply (arrow with broken lines).
- When P_1 releases the resource, it tells the coordinator, and then coordinator replies OK to P_2 . The request from P_2 is removed from the queue.

Limitation: Single coordinator, single point of failure. **How this can be fixed?**

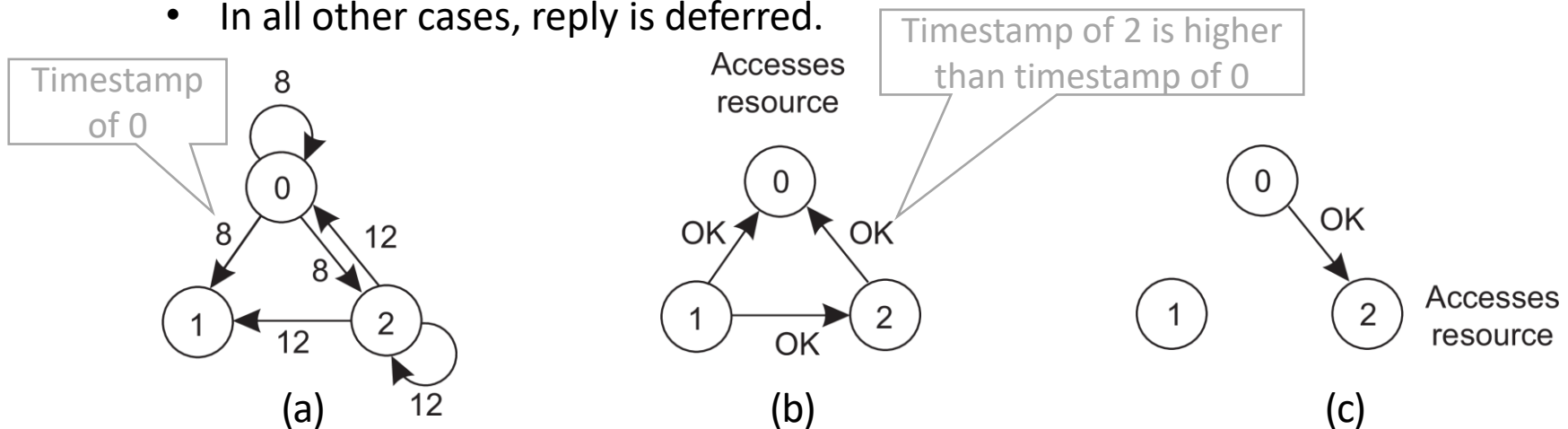
Mutual exclusion: distributed (Ricart & Agrawala)

(Section 5.3.3, in the textbook)

Prerequisite: There is no central coordinator. A process maintains a logical clock (Lamport's algorithm will be covered later in the term). A request to access a shared resource is sent to all processes. A request from a process contains a timestamp, its current (logical) time.

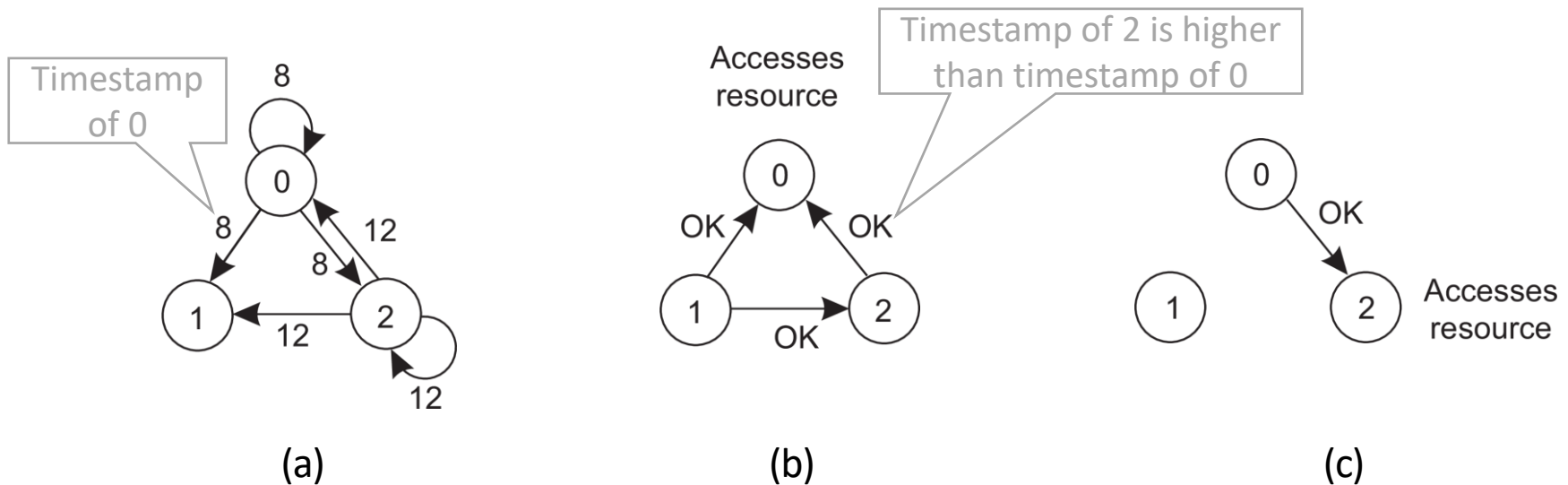
A process returns a response (i.e., replies OK) to a request only when:

- The receiving process has no interest in the shared resource; or
- The receiving process is waiting for the resource, but has lower priority (known through comparison of timestamps included in a request - lowest timestamp wins).
- In all other cases, reply is deferred.



- Two processes P_0 and P_2 want to access a shared resource at the same moment. P_1 does not want to access the shared resource.
- Process P_0 has the lowest timestamp, so it wins.
- When process P_0 is done, it sends an OK, so P_2 can now go ahead.

Mutual exclusion: distributed (Ricart & Agrawala)

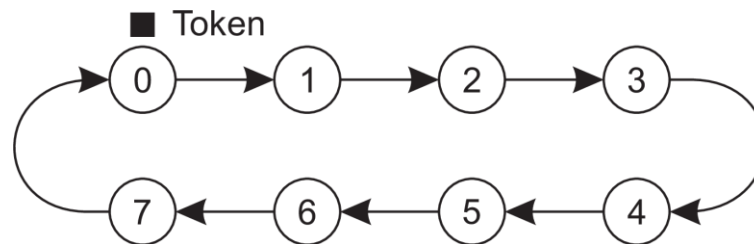


- In general, mutual exclusion is guaranteed without deadlock or starvation.
- Limitation: If any process crashes, it will fail to respond to requests. This silence will be interpreted (incorrectly) as denial of permission, thus blocking all subsequent attempts by all processes.

Mutual exclusion: token ring algorithm

(Section 5.3.4, in the textbook)

- Organize processes in a (logical) ring, and let a token be passed between them (in a predefined direction). The node that holds the token is allowed to enter the critical region (if it wants to).
- An overlay (virtual) network constructed as a logical ring with a circulating token.



- Since the token circulates among the processes in a well-defined order, starvation cannot occur.
- Limitation: If the token is ever lost, it must be regenerated. **But how do processors know that the token is lost?**

Mutual exclusion: token ring algorithm

- Messages per entry/exit (section 5.3.4, note 5.5, in the textbook):
 - With the token ring algorithm, the number is variable.
 - If every process constantly wants to enter a critical region, then each token pass will result in one entry and exit, for an average of one message per critical region entered.
 - At the other extreme, the token may sometimes circulate for hours without anyone being interested in it. In this case, the number of messages per entry into a critical region is unbounded.

Mutual exclusion: decentralized

(Section 5.3.5, in the textbook)

Voting-based algorithm:

- Assume every resource is replicated N times, with each replica having its own coordinator. Approval for shared resource access requires a majority vote from $m > N/2$ coordinators. (Assume a coordinator always responds immediately to a request - permission granted or denied.)
- A requesting process requires sending N messages to all replicas/coordinators, and receives N responses from coordinators. If it does not get a majority vote, the requesting process will have to release (at most) $N/2$ votes. If it did get enough votes, it will have to send an additional N release messages later on. A process may need to go through $k \geq 1$ attempts before granted permission.
- When a coordinator crashes, assuming it will recover quickly, but will have forgotten about permissions it had granted. As a consequence, it may incorrectly grant permission again to another process after its recovery.

Limitations: Can not guarantee fairness and may lead to starvation.

Mutual exclusion

(Section 5.3.4, note 5.5, in the textbook)

Algorithm	Messages per entry/exit	Delay before entry (in #messages)
Centralized	3	2
Distributed	$2(N - 1)$	$2(N - 1)$
Token ring	$1, \dots, \infty$	$0, \dots, N - 1$
Decentralized	$2kN + (k - 1)N/2 + N, k = 1, 2, \dots$	$2kN + (k - 1)N/2$

How to derive the above complexity of the decentralized/voting-based algorithms?
Hint: Three types of messages are communicated. A process may need to go through $k \geq 1$ attempts before granted permission to access the shared resource.

Gossip-based coordination, data dissemination (also called epidemic protocol)

Paper: Márk Jelasity, Spyros Voulgaris, Rachid Guerraoui, Anne-Marie Kermarrec, and Maarten van Steen. 2007. Gossip-based peer sampling. ACM Trans. Comput. Syst. 25, 3 (August 2007)

The goal is to spread information in a large-scale distributed system, often using node-local information only.

Here, the key idea is that there is no central coordinator, initially nodes have no knowledge of the network topology. Nodes interact in a peer-to-peer manner.

Anti-entropy message propagation model (section 4.4.3, in the textbook):

- Nodes exchange information in rounds. In each round, a node P selects another node Q from the system at random. (Variations include P selecting multiple random nodes, P ignoring previously selected nodes.)
- Pull: P only pulls in new updates from Q .
- Push: P only pushes its own updates to Q .
- Push-pull: P and Q send updates to each other. For push-pull, it takes $O(\log(N))$ rounds to disseminate updates to all N nodes (round = when every node has taken the initiative to start an exchange).

Application examples of gossip protocol (section 5.5, in the textbook):

a) Aggregating information, b) peer-sampling, c) overlay construction, and d) [Apache Cassandra distributed DB](#).